crypto: Hashing, HMAC, Random & Encryption

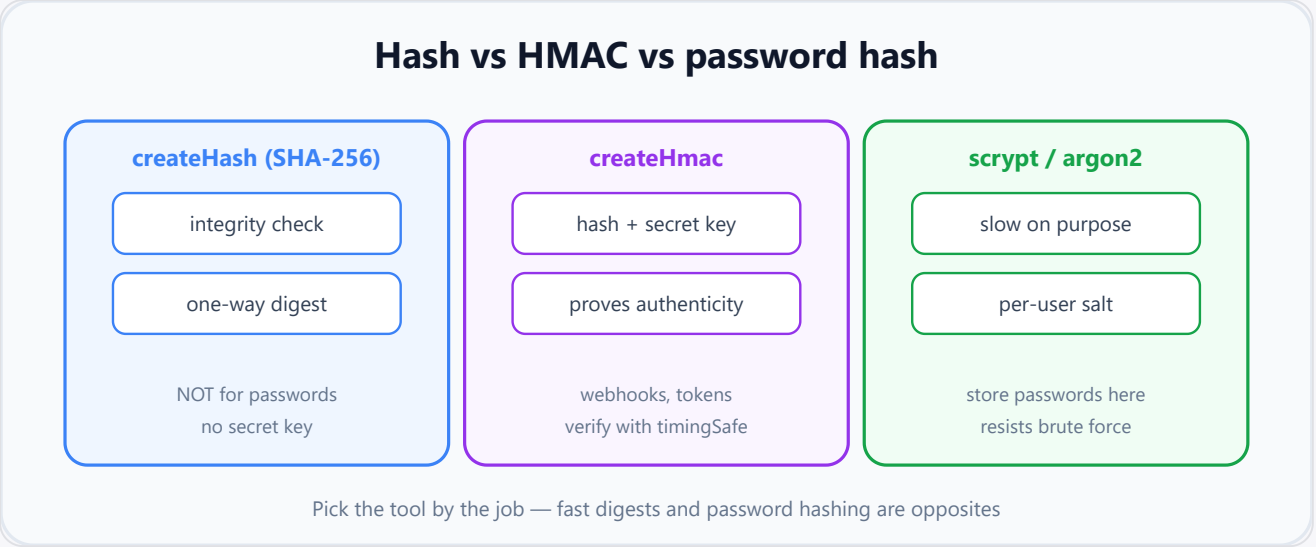
Why it matters

Security bugs are the most expensive kind of bug, and cryptography is the one area where "it looks like it works" is worthless — a broken scheme runs perfectly and encrypts nothing meaningful. Node's built-in `crypto` module gives you battle-tested primitives (backed by OpenSSL) for hashing, message authentication, secure random generation, password storage, and encryption. Knowing which primitive solves which problem — and, just as important, which ones must *never* be used for a given job — is core backend competence.

Interviewers lean on `crypto` because it separates people who reach for the right tool from those who "just SHA-256 the password." The consequences of getting it wrong are concrete: leaked credentials, forgeable tokens, predictable IDs, timing attacks. You do not need to invent cryptography — you need to use these primitives correctly and know their boundaries. This topic is about exactly that judgement.

The concept, simply

Four distinct jobs, four distinct tools. **Hashing** (`createHash`, e.g. SHA-256) turns any input into a fixed-size, one-way digest — used for integrity checks and deduplication, *not* secrecy. **HMAC** (`createHmac`) combines a hash with a secret key to prove a message came from someone who holds the key — used to sign and verify webhooks and tokens. **Secure randomness** (`randomBytes`, `randomUUID`) produces unpredictable values for tokens, salts, and IDs. **Password hashing** (`scrypt`, or argon2/bcrypt via libraries) is deliberately slow and salted, purpose-built to store passwords. **Encryption**



Match the primitive to the job: fast digests for integrity, HMAC for authenticity, slow salted hashing for passwords. They are not interchangeable.

How it works (deep)

A **cryptographic hash** is deterministic (same input, same digest), fixed-size (SHA-256 is always 32 bytes), one-way (you cannot reverse the digest to the input), and collision-resistant (finding two inputs with the same digest is infeasible). This makes it ideal for verifying that a file or message has not changed. It is disastrous for passwords, though, precisely *because* it is fast: an attacker with a leaked digest can try billions of guesses per second, and identical passwords produce identical digests, so rainbow tables crack them wholesale. Speed is a feature for integrity and a vulnerability for passwords.

HMAC fixes a subtle problem with plain hashing for authentication. You cannot simply hash `secret + message` to sign it — that construction is vulnerable to length-extension attacks against Merkle–Damgård hashes. HMAC's specific nested construction (hashing the key with inner and outer pads) is proven secure. When you receive a signed webhook, you recompute the HMAC over the raw body with your shared secret and compare it to the sender's signature. Crucially, that comparison must be **constant-time** (`crypto.timingSafeEqual`): a naive `===` returns as soon as bytes differ, leaking timing information that lets an attacker recover the signature byte by byte.

Secure randomness comes from the OS's cryptographically secure RNG via `crypto.randomBytes` and `crypto.randomUUID`. This is categorically different from

security guarantee downstream collapses.

Password hashing inverts the speed logic on purpose. `scrypt` (built in), `argon2`, and `bcrypt` are *slow* and memory-hard, with a tunable cost factor, and each password gets a unique random **salt** so identical passwords produce different stored hashes and rainbow tables are useless. You store the salt alongside the hash; on login you re-derive with the same salt and compare in constant time. The slowness is the whole point — it caps how many guesses an attacker can make per second.

Encryption is for data you must read back, unlike hashing. Use **AES-256-GCM**, an authenticated cipher: it needs a 32-byte key and a unique **IV** (initialisation vector) per message, and it produces both ciphertext and an authentication tag that detects tampering on decryption. The cardinal rule is *never reuse an IV with the same key* — doing so with GCM catastrophically breaks confidentiality and authenticity. Generate a fresh random IV for every encryption and store it with the ciphertext (it is not secret).

Code examples

Hashing for integrity, and HMAC for verifying a signed webhook in constant time:

JAVASCRIPT Copy

```
import { createHash, createHmac, timingSafeEqual } from 'node:crypto';

// Integrity digest of a payload (NOT for passwords):
const digest = createHash('sha256').update('file contents').digest('hex');

// Verify a webhook signature safely:
function verifyWebhook(rawBody, signatureHex, secret) {
  const expected = createHmac('sha256', secret).update(rawBody).digest();
  const got = Buffer.from(signatureHex, 'hex');
  // Length check first — timingSafeEqual throws on length mismatch.
  return got.length === expected.length && timingSafeEqual(got, expected);
}
```

Generating unpredictable tokens and IDs the secure way:

```
import { randomBytes, randomUUID, randomInt } from 'node:crypto';

const sessionToken = randomBytes(32).toString('hex'); // 256-bit token
const requestId = randomUUID();                       // RFC 4122 v4
const otp = randomInt(0, 1_000_000).toString().padStart(6, '0'); // secure
// NEVER: Math.random() for any of these — it is predictable.
```

Storing and verifying a password with the built-in `scrypt` (slow + salted):

JAVASCRIPT Copy

```
import { randomBytes, scrypt, timingSafeEqual } from 'node:crypto';
import { promisify } from 'node:util';

const scryptAsync = promisify(scrypt);

async function hashPassword(password) {
  const salt = randomBytes(16);
  const key = await scryptAsync(password, salt, 64); // 64-byte derived ke
  return `${salt.toString('hex')}:${key.toString('hex')}`; // store both
}

async function verifyPassword(password, stored) {
  const [saltHex, keyHex] = stored.split(':');
  const salt = Buffer.from(saltHex, 'hex');
  const expected = Buffer.from(keyHex, 'hex');
  const actual = await scryptAsync(password, salt, expected.length);
  return timingSafeEqual(actual, expected); // constant-time compare
}
```

Authenticated encryption with AES-256-GCM (fresh IV every time, store the tag):

JAVASCRIPT Copy

```
import { randomBytes, createCipheriv, createDecipheriv } from 'node:crypto

const KEY = randomBytes(32); // 256-bit key — in real code load from a sec
```

```
  const cipher = createCipheriv('aes-256-gcm', KEY, iv);
  const enc = Buffer.concat([cipher.update(plaintext, 'utf8'), cipher.fina
  const tag = cipher.getAuthTag();                    // detects tampering
  return { iv: iv.toString('hex'), tag: tag.toString('hex'), data: enc.toS
}

function decrypt({ iv, tag, data }) {
  const decipher = createDecipheriv('aes-256-gcm', KEY, Buffer.from(iv, 'h
  decipher.setAuthTag(Buffer.from(tag, 'hex'));      // throws if tampered
  return Buffer.concat([
    decipher.update(Buffer.from(data, 'hex')),
    decipher.final(),
  ]).toString('utf8');
}
```

Common mistakes & gotchas

- **Hashing passwords with SHA-256/MD5.** Fast hashes are trivially brute-forced. Passwords need slow, salted `scrypt`/argon2/bcrypt. This is the single most common crypto mistake.
- **Comparing secrets with `===`.** String/Buffer equality short-circuits on the first differing byte, leaking timing. Use `crypto.timingSafeEqual` on equal-length buffers.
- **Using `Math.random()` for anything security-sensitive.** It is predictable. Tokens, salts, IVs, OTPs, and IDs must come from `crypto.randomBytes`/`randomUUID`.
- **Reusing an IV (nonce) with AES-GCM.** Nonce reuse under the same key destroys GCM's security entirely. Generate a fresh random IV per message and store it with the ciphertext.
- **Confusing encoding with encryption.** Base64 is not encryption — it is reversible by anyone. Encoding hides nothing.
- **Encrypting without authentication.** Plain AES-CBC without a MAC lets attackers tamper undetected. Prefer AES-GCM (built-in authentication) and always verify the tag on decrypt.

- **Hardcoding keys in source.** Keys in the repo are compromised the moment the repo is. Load them from environment/secret managers and rotate them.
- **Rolling your own crypto.** Custom schemes are almost always broken. Use standard primitives with standard modes.

Best practices

- **Passwords → `scrypt`/argon2/bcrypt with a unique per-user salt,** never a plain hash.
- **Compare all secrets with `timingSafeEqual`** on equal-length buffers.
- **Generate every token, salt, IV, and ID with `crypto.randomBytes`/`randomUUID`.**
- **Encrypt with AES-256-GCM,** a fresh IV per message, and always verify the auth tag on decrypt.
- **Store the salt/IV/tag alongside the ciphertext** — they are not secret, and you need them to verify/decrypt.
- **Keep keys out of source;** load from a secret manager and support rotation.
- **Verify webhooks with HMAC over the raw body** (not the parsed/re-serialised body) and constant-time comparison.
- **Use standard primitives and modes; do not invent your own.**

⭐ Interview lens

> **Q: Why can't you store passwords with SHA-256?** Because SHA-256 is fast and unsalted by default: an attacker with the digests can try billions of guesses per second, and identical passwords hash identically, so rainbow tables crack them en masse. Passwords need a deliberately slow, salted, memory-hard function like `scrypt`, argon2, or bcrypt.

> **Q: What is the difference between a hash and an HMAC?** A plain hash is keyless — anyone can compute it, so it proves integrity but not origin. An HMAC mixes in a secret key with a proven construction, so a valid HMAC proves the message came

Q: Why use `timingSafeEqual` instead of `===` for a signature? `===` returns as soon as two bytes differ, so the comparison time reveals how many leading bytes matched. An attacker can exploit that timing to reconstruct the secret byte by byte. `timingSafeEqual` always compares the full length in constant time.

Q: What breaks if you reuse an IV with AES-GCM? Nonce reuse under the same key is catastrophic for GCM: it leaks the XOR of plaintexts and, worse, allows forging the authentication tag, breaking both confidentiality and integrity. Every message needs a fresh, unique IV.

Q: Is base64 a form of encryption? No. Base64 is a reversible encoding with no key — anyone can decode it. It provides zero confidentiality; it only makes binary data safe to transport as text.

Q: Why not `Math.random()` for a session token? `Math.random()` is a fast PRNG seeded predictably and not designed for security; its output can be predicted, which would let an attacker guess valid tokens. Security-sensitive randomness must come from `crypto.randomBytes` / `randomUUID`, backed by the OS CSPRNG.

Cheat-card

- Integrity → `createHash('sha256')` (fast, keyless, one-way).
- Authenticity → `createHmac('sha256', secret)` + `timingSafeEqual`.
- Passwords → `scrypt` /argon2/bcrypt, unique salt, slow on purpose. Never SHA/MD5.
- Randomness → `randomBytes` / `randomUUID` / `randomInt`. Never `Math.random()`.
- Encryption → AES-256-GCM, fresh IV per message, verify the auth tag.

- Store salt/IV/tag next to ciphertext (not secret); keep *keys* in a secret manager.
- Base64 = encoding, not encryption.
- Compare secrets in constant time; never `==`.
- Use standard primitives — never roll your own.

QUICK CHECK — no marks, just for you.

1. Why should passwords never be stored with a plain SHA-256 hash?

- ○ SHA-256 output is reversible, so anyone can recover the original password
- ○ SHA-256 is fast and unsalted, so attackers can try billions of guesses per second and rainbow tables crack identical passwords wholesale
- ○ SHA-256 produces a different digest each run, so verification is impossible
- ○ SHA-256 is too slow, making every login request time out

Check answer

2. Why must you use crypto.timingSafeEqual instead of === when comparing a signature?

- ○ === cannot compare Buffers at all, so it always returns false
- ○ timingSafeEqual is simply faster than === for long strings
- ○ === returns as soon as two bytes differ, leaking timing that lets an attacker reconstruct the secret byte by byte; timingSafeEqual compares in constant time
- ○ === decodes the buffers to UTF-8 first, corrupting binary signatures

Check answer

3. What happens if you reuse an IV (nonce) with the same key under AES-256-GCM?

Nothing: the IV is not secret and reusing it is a common optimization

- Decryption becomes slower but the data stays secure
- The ciphertext simply becomes larger by one block
- It is catastrophic: it leaks the XOR of plaintexts and allows forging the authentication tag, breaking both confidentiality and integrity

Check answer

Previous
Buffer: Working with Binary Data

Next
Timers: setTimeout, setInterval, setImmediate & the Event Loop

LEARN MORE

ARTICLES & DOCS

- Crypto | Node.js Documentation
  nodejs.org
- Web Crypto API | Node.js Documentation
  nodejs.org
- Web Crypto API | MDN
  developer.mozilla.org

VIDEOS



7 Cryptography Concepts EVERY Developer Should Know
Fireship

Hand-picked references for this lesson.